

COMPUTER VISION LABS 1-6

Complete Exam Notes + Generalized Python/OpenCV Code

Designed for your lab exam Simple theory, topic-wise code, when to use each method, parameter meanings, common mistakes, comparisons, MCQ facts, and a final printable cheat sheet.

Coverage: Lab Manual 1 through Lab Manual 6

Special inclusion: Lab 2 Photometric / Point Transformations

Format: Open-book, closed-internet revision guide

Prepared for: Syeda Sara Ali

Study flow

Pixels → point operations → geometry → features → segmentation → advanced detection and matching

How to Use These Notes

These notes follow the actual six-lab sequence implied by your syllabus: Lab 1 foundations, Lab 2 photometric transformations, Lab 3 geometric transformations, Lab 4 feature extraction, Lab 5 segmentation, and Lab 6 wavelets, boundaries, Hough transform, and SIFT.

Exam need	Where to look
Understand a concept	Read the definition, intuition, steps, and comparison.
Write code	Use the generalized template and replace only file path or listed parameters.
Choose a method	Use the “When to use” and decision tables.
Prepare MCQs	Revise bold facts, preservation rules, formulas, and common mistakes.
Closed internet	Print the final cheat-sheet section and the parameter reference.

Important numbering note Some earlier summaries shifted lab numbers. This document fixes the sequence by placing Photometric Transformations in Lab 2 and Geometric Transformations in Lab 3.

Universal Setup Used Throughout

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

PATH = "image.jpg"
image = cv2.imread(PATH)
if image is None:
    raise FileNotFoundError(f'Could not read: {PATH}')

rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
h, w = image.shape[:2]

def show(img, title="Image", cmap=None):
    plt.figure(figsize=(7, 5))
    plt.imshow(img, cmap=cmap)
    plt.title(title)
    plt.axis("off")
    plt.tight_layout()
    plt.show()
```

Color rule OpenCV reads a color image as BGR. Matplotlib expects RGB. Convert BGR to RGB only for Matplotlib display. Most OpenCV processing functions can continue using the original BGR image.

Lab 1 - Computer Vision and Basic Image Operations

1.1 What Computer Vision Covers

Computer Vision (CV) is the field that enables computers to obtain useful meaning from images and videos. Digital Image Processing (DIP) mainly improves or transforms images, while CV often goes further and makes a decision such as identifying an object, locating it, or understanding a scene.

Task	Simple meaning	Example
Image processing	Change or improve pixels	Blur noise or enhance contrast
Feature extraction	Create a compact description	Edges, corners, HOG or SIFT
Classification	Assign one label to an image	Cat versus dog
Object detection	Find and label objects	Car boxes in a street image
Segmentation	Assign a region or class to pixels	Separate tumor from tissue
Tracking	Follow an object over frames	Track a vehicle in video

1.2 Digital Images and Coordinates

A grayscale image can be represented as $f(x, y)$, where x is the horizontal column, y is the vertical row, and $f(x, y)$ is the intensity. A digital image has discrete coordinates and discrete intensity values. An 8-bit grayscale pixel normally ranges from 0 (black) to 255 (white). A color image commonly has three channels.

Term	Meaning
Pixel	Smallest addressable image element.
Resolution	Width $\times$ height, such as 1920×1080 .
Channel	One component of color, such as B, G, or R.
Grayscale	One intensity channel.
RGB	Red, Green, and Blue color model.
BGR	OpenCV default channel order.
Dynamic range	Range between lowest and highest representable intensity.
ROI	Region of interest, a selected part of the image.

Coordinate warning NumPy indexing is `image[row, column] = image[y, x]`. OpenCV points are usually written as (x, y) . Mixing these conventions is a common exam and coding mistake.

1.3 Digital Image Processing System and Fundamental Steps

Step	What it does
1. Acquisition	Capture or load an image, often with light preprocessing.
2. Enhancement	Make useful details easier to see.
3. Restoration	Recover an image using a degradation model.
4. Color processing	Work with color spaces and color information.
5. Wavelets / multiresolution	Analyze information at multiple scales.
6. Compression	Reduce storage or transmission size.

Step	What it does
7. Morphology	Process shapes using a structuring element.
8. Segmentation	Divide an image into meaningful regions.
9. Representation and description	Convert regions into boundaries or feature vectors.
10. Recognition	Assign labels or identities.
Knowledge base	Guides and controls other stages using prior information.

1.4 Essential Libraries: What to Use

Library	Best use
OpenCV (cv2)	General image/video I/O, transforms, filtering, features, segmentation.
NumPy	Array mathematics, masks, reshaping, matrices.
Matplotlib	Notebook-friendly display and plots.
Pandas	Tabular pixel statistics and analysis.
scikit-image	HOG, LBP, scientific image algorithms.
Pillow	Simple file conversion, drawing, resizing, metadata.
PyTorch / TorchVision	Deep-learning training, models, datasets, augmentation.
TensorFlow / Keras	Deep-learning models and deployment workflows.
Detectron2	Advanced object detection and instance segmentation.
OpenVINO	Optimized inference, especially on Intel hardware.

1.5 Basic Operations: Generalized Code and Use

Read, Validate, and Display

```
image = cv2.imread(PATH)
if image is None:
    raise FileNotFoundError(PATH)

rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
show(rgb, "Original")
```

Use at the start of nearly every task. Validation prevents later errors caused by a wrong file name or path.

Grayscale Conversion

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
show(gray, "Grayscale", cmap="gray")
```

Use before intensity-based thresholding, most edge detectors, histogram equalization, or when color is not needed. It reduces three channels to one and lowers computation.

Resize While Preserving Aspect Ratio

```
scale = 0.5
resized = cv2.resize(image, None, fx=scale, fy=scale,
                     interpolation=cv2.INTER_AREA)
```

Use INTER_AREA when shrinking and INTER_LINEAR or INTER_CUBIC when enlarging. Using equal fx and fy preserves the aspect ratio.

Gaussian Blur

```
kernel_size = (5, 5)    # both values must be positive odd numbers
sigma = 0
blurred = cv2.GaussianBlur(image, kernel_size, sigma)
```

Use to suppress noise before Canny, thresholding, or segmentation. A larger kernel removes more detail. Excessive blur destroys small objects and edges.

Crop a Region of Interest

```
x1, y1, x2, y2 = 100, 50, 400, 300
crop = image[y1:y2, x1:x2] # rows first, then columns
```

Use to focus processing on a selected area. The end indices are excluded. Ensure coordinates remain inside the image.

Write Text

```
output = image.copy()
cv2.putText(output, "Label", (30, 60),
            cv2.FONT_HERSHEY_SIMPLEX, 1.2,
            (0, 0, 255), 2, cv2.LINE_AA)
```

The position is the text baseline origin (x, y). The color is BGR because OpenCV is drawing on a BGR image.

Pixel Statistics with Pandas

```
import pandas as pd
pixels = image.reshape(-1, 3)
df = pd.DataFrame(pixels, columns=["B", "G", "R"])
print(df.describe())
```

Use to inspect count, mean, standard deviation, quartiles, minimum, and maximum for each channel. This is descriptive analysis, not a visual feature extractor by itself.

Binary Thresholding

```
threshold_value = 100
max_value = 255
_, binary = cv2.threshold(gray, threshold_value, max_value,
                          cv2.THRESH_BINARY)
```

Pixels greater than the threshold become max_value; all other pixels become 0. Use when foreground and background have clearly different intensities.

Image Blending: Addition vs Weighted Blending

```
# Images must have the same height, width, and channel count
bright_sum = cv2.add(image1, image2) # saturated addition, capped at 255

alpha = 0.6
beta = 0.4
blend = cv2.addWeighted(image1, alpha, image2, beta, gamma=0)
```

Use cv2.add for direct saturated addition. Use addWeighted for controlled blending, overlays, or transparency. NumPy uint8 addition can wrap around, so prefer OpenCV for image arithmetic.

Global Histogram Equalization

```
equalized = cv2.equalizeHist(gray)
```

Use on a grayscale image when global contrast is poor. It remaps intensities using the cumulative histogram. It may amplify noise or look unnatural, so CLAHE is often better for locally uneven contrast.

Lab 2 - Photometric and Point Transformations

Photometric transformations change pixel values, not pixel positions. A point transformation applies the same intensity function independently to every pixel: $s = T(r)$. Here r is the input intensity and s is the output intensity.

2.1 Photometric vs Geometric

Type	Changes	Examples
Photometric / point	What a pixel looks like	Negative, log, gamma, contrast stretching, histogram processing
Geometric	Where a pixel is located	Translation, rotation, scaling, shear, perspective

2.2 Intensity Negative

A negative reverses brightness: $s = L - 1 - r$. For an 8-bit image, $s = 255 - r$. Bright areas become dark and dark areas become bright.

```
negative = 255 - gray
# Equivalent OpenCV form: negative = cv2.bitwise_not(gray)
```

Use when light details are easier to observe on a dark background, for example some X-ray or photographic negative analysis.

2.3 Log Transformation

The log transform expands low intensities and compresses high intensities: $s = c \log(1 + r)$. It reveals dark details while controlling a very large dynamic range.

```
img = gray.astype(np.float32)
c = 255.0 / np.log1p(img.max())
log_img = c * np.log1p(img)
log_img = np.clip(log_img, 0, 255).astype(np.uint8)
```

When to use Use log transformation for Fourier magnitude spectra or images with a few extremely bright values and many dark values. Convert to float first to avoid arithmetic problems.

2.4 Gamma or Power-Law Transformation

Gamma transformation uses $s = c r^{\gamma}$ after normalizing r to $[0, 1]$. Gamma controls the curve. In this standard implementation, $\gamma < 1$ brightens midtones and $\gamma > 1$ darkens midtones.

```
gamma = 0.5
normalized = gray.astype(np.float32) / 255.0
gamma_img = np.power(normalized, gamma)
gamma_img = np.clip(gamma_img * 255, 0, 255).astype(np.uint8)
```

Gamma	Effect	Typical use
$\gamma < 1$	Brightens dark and midtone areas	Underexposed image
$\gamma = 1$	No change	Identity check
$\gamma > 1$	Darkens midtones	Overly bright or washed image

Fast Gamma with a Lookup Table

```
gamma = 0.5
lut = np.array([((i / 255.0) ** gamma) * 255
                 for i in range(256)], dtype=np.uint8)
gamma_img = cv2.LUT(gray, lut)
```

A lookup table computes each output value once and then maps all pixels efficiently. This is useful for video or repeated processing.

2.5 Piecewise-Linear Transformations

A piecewise transform uses different linear rules for different intensity ranges. It is useful when one known range contains important information.

Contrast Stretching

```
r1, r2 = 70, 180
stretched = np.interp(gray, [0, r1, r2, 255],
                          [0, 0, 255, 255])
stretched = stretched.astype(np.uint8)
```

Use when useful intensities occupy a narrow interval. Values below $r1$ become dark, values above $r2$ become bright, and the middle range is expanded.

Intensity-Level Slicing

```
low, high = 100, 160
sliced = np.zeros_like(gray)
sliced[(gray >= low) & (gray <= high)] = 255
```

Use to highlight a selected intensity band, such as a material, tissue, or region whose intensity range is known. A second version can preserve the background instead of setting it to zero.

2.6 Histograms and Contrast

A histogram counts pixels at each intensity. A dark image is concentrated toward the left, a bright image toward the right, a low-contrast image occupies a narrow range, and a high-contrast image is widely spread.

```
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])
plt.plot(hist)
plt.xlim([0, 256])
plt.title("Grayscale Histogram")
plt.xlabel("Intensity")
plt.ylabel("Pixel count")
plt.show()
```

Histogram Equalization

```
equalized = cv2.equalizeHist(gray)
```

The mapping is based on the cumulative distribution function (CDF). It generally spreads intensities and improves global contrast, but the resulting histogram is not guaranteed to be perfectly flat in a discrete image.

CLAHE: Local Contrast Enhancement

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
clahe_img = clahe.apply(gray)
```

CLAHE equalizes small tiles and limits amplification. Use it for uneven lighting or medical images where global equalization over-enhances noise. Higher clipLimit produces stronger contrast and potentially more noise.

2.7 Point Transformation Decision Guide

Situation	Use	Reason
Huge dynamic range	Log	Compresses bright values and expands dark values.
Dark image	$\text{Gamma} < 1$	Brightens midtones.
Image too bright	$\text{Gamma} > 1$	Darkens midtones.
Known intensity band	Piecewise or slicing	Direct control of selected range.
Globally low contrast	Histogram equalization	Automatic global remapping.
Uneven local contrast	CLAHE	Local enhancement with clipping.

Lab 3 - Geometric Transformations

A geometric transformation changes spatial coordinates: $(x, y) \rightarrow (x', y')$. Pixel locations move, and interpolation estimates output values where transformed coordinates do not fall exactly on integer pixels.

3.1 Image Coordinates, Mapping, and Interpolation

- Image origin is at the top-left; x increases rightward and y increases downward.
- Forward mapping sends each input point to an output point and may leave holes.
- Inverse mapping asks where each output pixel came from and is commonly used by warping functions.
- Nearest-neighbor interpolation is fastest and preserves labels or masks.
- Bilinear interpolation is a good general choice for natural images.
- Bicubic interpolation is slower but can be smoother when enlarging.

3.2 Linear, Affine, and Projective Transformations

Class	Matrix idea	Preserves	Examples
Linear	$2 \times 2, p' = Ap$	Origin, lines, parallelism	Scale, rotate, reflect, shear
Affine	$2 \times 3, p' = Ap + b$	Lines, collinearity, parallelism, ratios on a line	Linear transforms + translation
Projective	3×3 homography	Straight lines and collinearity	Perspective warp

Correction for MCQs A general rotation preserves lengths and angles, but a general linear transformation does not. Therefore, do not mark “angles preserved” for all linear transformations.

3.3 Homogeneous Coordinates

A 2×2 linear matrix cannot translate because $A0 = 0$. Homogeneous coordinates add a third coordinate: (x, y) becomes $(x, y, 1)$. Translation then fits into a 3×3 matrix, making composition possible by matrix multiplication. OpenCV stores an affine transform in its first two rows as a 2×3 matrix.

3.4 Generalized Transformations

Translation

```
tx, ty = 100, 50
M = np.float32([[1, 0, tx],
                [0, 1, ty]])
translated = cv2.warpAffine(image, M, (w, h),
                             flags=cv2.INTER_LINEAR,
                             borderMode=cv2.BORDER_CONSTANT,
                             borderValue=(0, 0, 0))
```

Positive tx moves right; positive ty moves down. Increase output canvas size if content must not be cut off.

Rotation Around the Image Center

```
center = (w / 2, h / 2)
angle = 45 # positive is counter-clockwise in OpenCV display coordinates
scale = 1.0
M = cv2.getRotationMatrix2D(center, angle, scale)
rotated = cv2.warpAffine(image, M, (w, h))
```

Use for orientation correction or augmentation. A fixed canvas cuts corners. For a no-crop version, compute the rotated corner bounds and enlarge the output canvas.

Scaling

```
scale_x, scale_y = 1.5, 1.5
scaled = cv2.resize(image, None, fx=scale_x, fy=scale_y,
                    interpolation=cv2.INTER_LINEAR)
```

Equal scale factors preserve aspect ratio. Unequal values stretch the image. Use nearest-neighbor for class masks to avoid inventing label values.

Horizontal Shear

```
kx = 0.5
M = np.float32([[1, kx, 0],
               [0, 1, 0]])
new_w = int(w + abs(kx) * h)
sheared = cv2.warpAffine(image, M, (new_w, h))
```

Equation: $x' = x + kx \cdot y$ and $y' = y$. Use in augmentation or to model skew. A negative shear may require an added translation to keep coordinates positive.

Rigid Transformation

```
M = cv2.getRotationMatrix2D((w/2, h/2), 30, 1.0)
M[0, 2] += 100
M[1, 2] += 50
rigid = cv2.warpAffine(image, M, (w, h))
```

Rigid motion combines rotation and translation. The scale must remain 1.0, and there must be no shear, so distances and angles are preserved.

Affine Transform from Three Point Pairs

```
src = np.float32([[50, 50], [300, 50], [50, 300]])
dst = np.float32([[30, 80], [320, 40], [80, 320]])
M = cv2.getAffineTransform(src, dst)
affine = cv2.warpAffine(image, M, (w, h))
```

Three non-collinear point correspondences determine a 2D affine transformation. Use for planar alignment where parallel lines should stay parallel.

Perspective Transform from Four Point Pairs

```
src = np.float32([[0, 0], [w-1, 0],
                 [w-1, h-1], [0, h-1]])
dst = np.float32([[80, 40], [w-100, 0],
                 [w-30, h-60], [40, h-20]])
H = cv2.getPerspectiveTransform(src, dst)
warped = cv2.warpPerspective(image, H, (w, h))
```

Four ordered point pairs determine a homography. Use for document rectification, planar viewpoint correction, panorama alignment, or perspective simulation.

3.5 Transformation Selection

Need	Method	OpenCV
Move only	Translation	warpAffine
Rotate or rotate + scale	getRotationMatrix2D + warpAffine	2×3 matrix
Skew or general planar affine alignment	getAffineTransform + warpAffine	Three point pairs
Correct document perspective	getPerspectiveTransform + warpPerspective	Four point pairs
Estimate perspective from noisy feature matches	findHomography with RANSAC	Four or more matches

Lab 4 - Feature Extraction, Filtering, Edges, and Corners

Feature extraction converts raw pixels into a smaller and more meaningful description. A good feature is distinctive, reasonably robust to noise or lighting, and suitable for matching or learning.

4.1 Local vs Global Features

Type	Meaning	Examples	Use
------	---------	----------	-----

Type	Meaning	Examples	Use
Local	Computed around selected locations	Corners, SIFT, ORB	Matching, stitching, recognition
Global	Describes an entire image or region	Color histogram, HOG, texture histogram	Classification and retrieval

4.2 Convolution and Filtering

Convolution slides a small matrix called a kernel across an image. At each location, corresponding values are multiplied and summed. Filters can smooth noise, sharpen detail, or measure intensity change.

```
kernel = np.ones((3, 3), np.float32) / 9.0
box_blur = cv2.filter2D(gray, ddepth=-1, kernel=kernel)

gaussian = cv2.GaussianBlur(gray, (5, 5), sigmaX=1.0)
```

Padding and stride OpenCV filter2D preserves image size by using a border rule. It does not have a strides argument. For true valid or strided convolution, crop the valid area or implement sampling separately. In CNN formulas: $\text{output} = \text{floor}((N + 2P - K)/S) + 1$.

Filter	Purpose	Main risk
Box blur	Fast average smoothing	Blurs edges strongly
Gaussian	Noise reduction with weighted neighborhood	Large sigma removes fine detail
Median	Excellent for salt-and-pepper noise	Can remove thin lines
Sobel / Scharr	First derivatives and edges	Sensitive to noise without smoothing
Laplacian	Second derivative and rapid changes	Very noise-sensitive

4.3 HOG: Histogram of Oriented Gradients

HOG describes local shape through gradient directions. The image is divided into cells, each cell builds an orientation histogram, neighboring cells form blocks, and block normalization reduces sensitivity to illumination.

```
from skimage.feature import hog

features, hog_vis = hog(
    gray,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    block_norm="L2-Hys",
    visualize=True,
    feature_vector=True
)
print(features.shape)
```

Use for pedestrian or shape-based classification, especially with a traditional classifier such as a linear SVM. Weaknesses include sensitivity to large pose changes and less flexibility than learned CNN features.

4.4 LBP: Local Binary Pattern

LBP compares neighboring intensities with the center pixel. A neighbor greater than or equal to the center receives 1; otherwise 0. The resulting binary pattern describes local texture, and a histogram becomes the feature vector.

```
from skimage.feature import local_binary_pattern

radius = 1
points = 8 * radius
lbp = local_binary_pattern(gray, points, radius, method="uniform")

bins = int(lbp.max() + 1)
hist, _ = np.histogram(lbp.ravel(), bins=bins,
    range=(0, bins), density=True)
```

Use for texture or face-description tasks. LBP is fast and somewhat robust to monotonic illumination changes, but noise near equal intensities can flip bits.

4.5 Color Histogram

```
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
hist = cv2.calcHist([hsv], [0, 1], None,
                    [30, 32], [0, 180, 0, 256])
cv2.normalize(hist, hist)
```

Use for retrieval or classification when color distribution matters more than spatial layout. HSV is often preferable because hue separates color from brightness. A histogram loses location information, so two differently arranged images can look similar to it.

4.6 Sobel, Canny, and LoG

Sobel Gradient

```
gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
magnitude = cv2.magnitude(gx.astype(np.float32), gy.astype(np.float32))
direction = np.arctan2(gy, gx)
```

Gx responds strongly to vertical edges, and Gy to horizontal edges. Use CV_64F so negative derivatives are not lost. Magnitude gives edge strength; direction gives orientation.

Canny

```
blurred = cv2.GaussianBlur(gray, (5, 5), 1.0)
edges = cv2.Canny(blurred, threshold1=50, threshold2=150)
```

1. Gaussian smoothing reduces noise.
2. Gradients estimate strength and direction.
3. Non-maximum suppression thins responses.
4. Double threshold and hysteresis retain connected weak edges and reject isolated noise.

Use Canny for thin, connected boundaries. Increasing both thresholds reduces detected edges. The lower threshold should be below the upper threshold; a common starting ratio is roughly 1:2 or 1:3, then tune for the image.

Laplacian of Gaussian Approximation

```
blurred = cv2.GaussianBlur(gray, (5, 5), 1.4)
lap = cv2.Laplacian(blurred, cv2.CV_64F, ksize=3)
lap_abs = cv2.convertScaleAbs(lap)
```

LoG first suppresses noise and then uses a second derivative. In theory, edges are identified by zero crossings. Displaying absolute Laplacian responses shows strong changes but is not itself explicit zero-crossing detection.

4.7 Histogram of Edge Directions / Intensity Gradients

```
angles = (np.degrees(np.arctan2(gy, gx)) + 360) % 360
mag = np.sqrt(gx**2 + gy**2)
mask = mag > 50
hist, bins = np.histogram(angles[mask], bins=8,
                          range=(0, 360), weights=mag[mask])
hist = hist / (hist.sum() + 1e-8)
```

Weighting orientation bins by gradient magnitude is more informative than counting all pixels equally. Use global HED/HIG for coarse shape or texture description; HOG adds cells and block normalization, so it retains more spatial structure.

4.8 Texture Energy and Contrast: Safer General Form

```
img_f = gray.astype(np.float32)
mean = cv2.blur(img_f, (5, 5))
mean_sq = cv2.blur(img_f**2, (5, 5))
variance = np.maximum(mean_sq - mean**2, 0)
local_contrast = np.sqrt(variance)
local_energy = cv2.blur(img_f**2, (5, 5))
```

Convert to float before squaring because uint8 arithmetic can overflow. Energy measures squared intensity in a neighborhood; local standard deviation is a practical contrast measure. For material classification, combine LBP, Gabor filters, or gray-level co-occurrence matrix features rather than relying on raw energy alone.

4.9 Corner Detection

Corners are locations where intensity changes strongly in two directions. They are stable reference points for tracking, matching, calibration, and motion estimation.

Harris Corner Detector

```
gray_f = np.float32(gray)
response = cv2.cornerHarris(gray_f, blockSize=2, ksize=3, k=0.04)
output = image.copy()
output[response > 0.01 * response.max()] = (0, 0, 255)
```

Shi-Tomasi Good Features to Track

```
corners = cv2.goodFeaturesToTrack(gray, maxCorners=100,
                                  qualityLevel=0.01,
                                  minDistance=10)
if corners is not None:
    corners = np.int32(corners).reshape(-1, 2)
```

Shi-Tomasi is usually the direct choice for tracking points. Harris provides a response map and requires thresholding.

Lab 5 - Image Segmentation

Segmentation partitions an image into meaningful, non-overlapping regions. A binary mask usually stores background as 0 and foreground as 255. Good segmentation depends on lighting, noise, color space, preprocessing, and parameters.

5.1 Main Families

Family	Core idea	Best fit
Thresholding	Separate intensities or colors	Simple objects and backgrounds
Edge-based	Use boundaries	Clear closed contours
Region-based	Grow similar connected pixels	Uniform regions with a reliable seed
Watershed	Flood from markers	Touching objects
Clustering	Group similar feature vectors	Color or texture grouping
Deep learning	Predict pixel labels	Complex semantic or instance tasks
Active contours	Evolve a curve toward a boundary	Boundary refinement

5.2 Thresholding

Global Threshold

```
_, mask = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)
```

Use only when one threshold works across the whole image.

Adaptive Threshold

```
mask = cv2.adaptiveThreshold(
    gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, blockSize=11, C=2
)
```

Use for uneven illumination, especially document images. `blockSize` must be odd and greater than 1. Increasing `C` makes the local threshold lower because `C` is subtracted from the local statistic.

Otsu Threshold

```
blur = cv2.GaussianBlur(gray, (5, 5), 0)
threshold_used, mask = cv2.threshold(
    blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
```

Otsu automatically chooses a threshold that minimizes within-class variance, equivalently maximizes between-class separation. It works best for roughly bimodal intensity distributions.

HSV Color Segmentation

```
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
lower = np.array([30, 50, 50], dtype=np.uint8)
upper = np.array([60, 255, 255], dtype=np.uint8)
mask = cv2.inRange(hsv, lower, upper)
```

Use for a distinctly colored object. OpenCV hue for 8-bit HSV is 0 to 179, not 0 to 360. Red wraps around the hue boundary and often needs two masks.

5.3 Region Growing

```
def region_growing(gray, seed_xy, tolerance=15):
    h, w = gray.shape
    sx, sy = seed_xy
    if not (0 <= sx < w and 0 <= sy < h):
        raise ValueError("Seed is outside the image")

    mask = np.zeros((h, w), np.uint8)
    visited = np.zeros((h, w), bool)
    stack = [(sx, sy)]
    seed_value = int(gray[sy, sx])

    while stack:
        x, y = stack.pop()
        if x < 0 or x >= w or y < 0 or y >= h or visited[y, x]:
            continue
        visited[y, x] = True
        if abs(int(gray[y, x]) - seed_value) <= tolerance:
            mask[y, x] = 255
            stack.extend([(x+1,y), (x-1,y), (x,y+1), (x,y-1)])
    return mask
```

Use when the region is connected and approximately uniform. The seed is passed as (x, y), but NumPy is indexed [y, x]. A fixed seed comparison may stop across gradual intensity change; comparing with the evolving region mean is a more adaptive variant.

5.4 Morphological Cleanup

```
kernel = np.ones((3, 3), np.uint8)
eroded = cv2.erode(mask, kernel, iterations=1)
dilated = cv2.dilate(mask, kernel, iterations=1)
opened = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
closed = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
```

Operation	Order / effect	Use
Erosion	Shrinks white foreground	Remove small white noise; separate thin connections
Dilation	Expands white foreground	Fill small gaps; connect nearby parts
Opening	Erosion then dilation	Remove isolated white noise
Closing	Dilation then erosion	Fill small black holes and gaps
Gradient	Dilation minus erosion	Approximate object boundary

5.5 Marker-Based Watershed

```
img = image.copy()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 0, 255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel, iterations=2)
sure_bg = cv2.dilate(opening, kernel, iterations=3)

dist = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist, 0.5 * dist.max(), 255, 0)
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)

_, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0
markers = cv2.watershed(img, markers)

result = img.copy()
result[markers == -1] = (0, 0, 255) # red in BGR
```

Use for touching foreground objects. Watershed requires reliable markers: background, confident foreground, and unknown regions. marker value -1 is the boundary. If shown in Matplotlib, convert result from BGR to RGB.

5.6 K-Means Color Segmentation

```

pixels = image.reshape(-1, 3).astype(np.float32)
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
            100, 0.2)
K = 4
compactness, labels, centers = cv2.kmeans(
    pixels, K, None, criteria, attempts=10,
    flags=cv2.KMEANS_PP_CENTERS
)
centers = np.uint8(centers)
segmented = centers[labels.ravel()].reshape(image.shape)

```

Use to group pixels by color. K must be selected, and clusters do not necessarily correspond to real objects because spatial location is ignored. Add normalized x and y features if spatial compactness is important.

5.7 Deep Learning and Active Contours

Method	Output	Examples	Requirement
Semantic segmentation	Class per pixel; same-class objects merge	U-Net, FCN, DeepLab	Labeled masks and compute
Instance segmentation	Separate mask per object instance	Mask R-CNN	Detailed instance annotations
Active contour	A curve fitted to a boundary	Snakes, Chan-Vese	Initialization near useful boundary

5.8 Segmentation Decision Guide

Image condition	Recommended start
Uniform lighting and clear intensity gap	Global threshold or Otsu
Uneven document lighting	Adaptive threshold
Distinct object color	HSV inRange
Uniform connected region and known seed	Region growing
Touching coins or cells	Distance transform + watershed
Several dominant colors	K-means
Complex real-world classes	Deep learning

Lab 6 - Wavelets, Boundary Detection, Hough Transform, and SIFT

6.1 Wavelet Transformation

Wavelets represent a signal or image using localized basis functions at different scales. Unlike a global Fourier representation, wavelets preserve both location and frequency-scale information, making them useful for non-stationary signals, denoising, compression, and texture features.

Transform	Meaning	Typical use
CWT	Many continuous scales and shifts	Detailed time-frequency analysis
DWT	Discrete dyadic scales and shifts	Compression, denoising, features
WPT	Decomposes approximation and detail branches	Richer frequency-band analysis
Inverse transform	Reconstructs from coefficients	Recover processed signal or image

2D Discrete Wavelet Transform

```
import pywt

img_f = gray.astype(np.float32) / 255.0
LL, (LH, HL, HH) = pywt.dwt2(img_f, "haar")

# LL: low-frequency approximation
# LH, HL, HH: directional detail subbands
reconstructed = pywt.idwt2((LL, (LH, HL, HH)), "haar")
```

Use LL as a coarse image and detail bands for edges or texture. For denoising, threshold small detail coefficients and then reconstruct. Multiple decomposition levels provide multi-resolution analysis.

6.2 Boundary Detection Summary

Method	Derivative idea	Result	Best use
Sobel	First derivative	Gradient magnitude and direction	Fast directional edges
Canny	First derivative pipeline	Thin connected binary edges	General boundary detection
LoG	Second derivative after smoothing	Zero crossings / strong responses	Blob-like and multi-scale changes

6.3 Hough Line Transform

The robust line parameterization is $\rho = x \cos(\theta) + y \sin(\theta)$, not slope-intercept $y = mx + b$, because vertical lines would require infinite slope. Every edge pixel votes for possible (ρ, θ) pairs. Peaks represent likely lines.

Probabilistic Hough Lines

```
edges = cv2.Canny(cv2.GaussianBlur(gray, (5, 5), 0), 50, 150)
lines = cv2.HoughLinesP(edges,
    rho=1,
    theta=np.pi/180,
    threshold=80,
    minLineLength=50,
    maxLineGap=10)

output = image.copy()
if lines is not None:
    for x1, y1, x2, y2 in lines[:, 0]:
        cv2.line(output, (x1, y1), (x2, y2),
            (0, 0, 255), 2)
```

Use HoughLinesP when line segments and endpoints are needed. threshold is the minimum vote count, minLineLength rejects short segments, and maxLineGap joins nearby collinear segments.

6.4 Hough Circle Transform

```
blur = cv2.medianBlur(gray, 5)
circles = cv2.HoughCircles(
    blur, cv2.HOUGH_GRADIENT,
    dp=1.2,
    minDist=30,
    param1=100,
    param2=30,
    minRadius=10,
    maxRadius=100
)

output = image.copy()
if circles is not None:
    circles = np.uint16(np.around(circles[0]))
    for x, y, r in circles:
        cv2.circle(output, (x, y), r, (0, 255, 0), 2)
        cv2.circle(output, (x, y), 2, (0, 0, 255), 3)
```

Use for coins, pupils, wheels, or circular parts. Lower param2 detects more circles but increases false positives. minDist prevents multiple nearby center detections. Restricting radius range improves reliability.

6.5 SIFT Feature Extraction

SIFT detects and describes distinctive local keypoints. It is designed to be invariant to scale and rotation and reasonably robust to illumination and viewpoint changes.

5. Build Gaussian scale space and Difference-of-Gaussians images.
6. Find extrema against 26 neighbors across location and scale.
7. Refine location and reject unstable low-contrast or edge-like points.
8. Assign one or more dominant local gradient orientations.
9. Build a 4×4 grid of orientation histograms with 8 bins: $4 \times 4 \times 8 = 128$ values.

```
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)

visual = cv2.drawKeypoints(
    image, keypoints, None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
print("Keypoints:", len(keypoints))
print("Descriptor shape:", None if descriptors is None else descriptors.shape)
```

The descriptor array normally has one 128-dimensional row per keypoint. Always handle the possibility that no descriptors are found.

6.6 SIFT Matching, Ratio Test, and RANSAC

```
img1 = cv2.imread("reference.jpg", cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread("scene.jpg", cv2.IMREAD_GRAYSCALE)

sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

if des1 is None or des2 is None:
    raise ValueError("Not enough texture for SIFT descriptors")

matcher = cv2.BFMatcher(cv2.NORM_L2)
knn = matcher.knnMatch(des1, des2, k=2)

good = []
for m, n in knn:
    if m.distance < 0.75 * n.distance: # Lowe ratio test
        good.append(m)

if len(good) >= 4:
    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1,1,2)
    dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1,1,2)
    H, inlier_mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
else:
    H, inlier_mask = None, None
```

The ratio test rejects ambiguous descriptor matches. RANSAC repeatedly estimates a model from small samples and keeps geometrically consistent inliers. At least four point pairs are needed for a homography, but more reliable inliers are strongly preferred.

6.7 SIFT vs ORB

Feature	SIFT	ORB
Descriptor	128-D floating point	Binary descriptor
Matcher norm	L2	Hamming
Strength	High matching robustness	Fast and lightweight
Use	Stitching, recognition, difficult scale changes	Real-time systems and limited compute

Integrated View: How the Six Labs Connect

10. Acquire and inspect the image.
11. Apply photometric correction if brightness or contrast is poor.
12. Apply geometric correction if the image is rotated, skewed, or perspective-distorted.
13. Reduce noise and calculate edges, textures, colors, corners, or keypoints.
14. Segment the scene if object regions are required.
15. Detect lines or circles, match SIFT features, or feed features into a classifier.

End-to-End Example: Document Rectification

- Lab 1: Load and grayscale the photograph.
- Lab 4: Blur and detect edges with Canny.
- Lab 6: Detect long page boundaries with Hough lines or contours.
- Lab 3: Order four page corners and apply a perspective transform.
- Lab 2: Apply adaptive thresholding or CLAHE to improve readability.

End-to-End Example: Match an Object in a Scene

- Lab 1: Load reference and scene images.
- Lab 4 / 6: Extract SIFT keypoints and descriptors.
- Lab 6: Match descriptors, apply ratio test, then RANSAC.
- Lab 3: Estimate homography and project reference corners into the scene.

Method Selection Master Table

Goal	Preferred method	Remember
Improve dark detail	$\text{Gamma} < 1$ or log	Log is especially useful for huge dynamic range
Correct uneven contrast	CLAHE	Clip limit controls amplification
Rotate or shift	Affine warp	2×3 matrix
Fix perspective	Homography	Four point pairs minimum
Remove Gaussian-like noise	Gaussian blur	Odd kernel size
Remove salt-and-pepper noise	Median filter	Preserves edges better than averaging
Fast edges	Sobel	Produces gradient, not automatically a binary edge map
Thin connected edges	Canny	Blur, gradient, NMS, hysteresis
Texture descriptor	LBP	Histogram of local binary patterns
Shape descriptor	HOG	Cell histograms + block normalization
Touching objects	Watershed	Marker quality is critical
Color groups	K-means	K must be selected; ignores space unless added
Detect lines	HoughLinesP	Operate on a clean edge map
Detect circles	HoughCircles	Tune param2 and radius range
Robust image matching	SIFT + ratio test + RANSAC	SIFT uses L2 distance

High-Yield MCQ Facts and Common Traps

- OpenCV color images are BGR; Matplotlib display is RGB.
- Image shape is (height, width, channels), not (width, height, channels).
- NumPy crop order is [y1:y2, x1:x2].
- An 8-bit grayscale image normally ranges from 0 to 255.
- $\gamma < 1$ brightens midtones; $\gamma > 1$ darkens them in the standard normalized formula.
- Histogram equalization uses a CDF but does not guarantee a perfectly uniform discrete histogram.
- Linear transformations cannot translate because the origin remains fixed.
- Affine transformations preserve parallelism; projective transformations may not.
- warpAffine uses a 2×3 matrix; warpPerspective uses a 3×3 matrix.
- Nearest-neighbor interpolation should be used for label masks.
- Gaussian kernel dimensions are normally odd and positive.
- Gx detects vertical edges; Gy detects horizontal edges.
- Canny stages are smoothing, gradient, non-maximum suppression, and hysteresis.
- Opening removes small white noise; closing fills small black holes.
- Otsu works best for approximately bimodal histograms.
- OpenCV HSV hue range is 0 to 179 for 8-bit images.
- Watershed marks boundaries with -1.
- Hough lines commonly use rho and theta, avoiding the vertical-line problem of slope-intercept form.
- SIFT descriptors have 128 values and use L2 distance.
- ORB descriptors are binary and use Hamming distance.
- A homography needs at least four point correspondences.
- RANSAC rejects geometrically inconsistent matches.

Corrections to Watch For

Incorrect shortcut	Correct statement
"Canny is always best."	Canny is strong for thin boundaries, but the best method depends on noise, texture, and task.
"Linear transforms preserve angles."	Only special linear transforms such as rotation and uniform scaling preserve angles.
"Histogram equalization makes a perfectly flat histogram."	Discrete equalization usually spreads values but may not be perfectly uniform.
"filter2D supports strides."	OpenCV filter2D has no strides parameter.
"Texture contrast is just a box-filtered image."	Local contrast is better represented by local standard deviation or range.
"Hough lines use $y = mx + b$ in practice."	OpenCV standard Hough uses rho-theta parameterization.
"A red watershed boundary is [255,0,0]."	On a BGR image, red is (0,0,255); [255,0,0] is blue.

Printable Code Cheat Sheet

Load and Prepare

```
import cv2, numpy as np, matplotlib.pyplot as plt
img = cv2.imread("image.jpg")
if img is None: raise FileNotFoundError("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
h, w = img.shape[:2]
```

Photometric

```
negative = 255 - gray
log_img = 255/np.log1p(gray.max()) * np.log1p(gray.astype(np.float32))
gamma = 0.5
gamma_img = np.uint8(255 * (gray.astype(np.float32)/255) ** gamma)
equalized = cv2.equalizeHist(gray)
clahe_img = cv2.createCLAHE(2.0, (8,8)).apply(gray)
```

Geometric

```
M = np.float32([[1,0,tx],[0,1,ty]])
translated = cv2.warpAffine(img, M, (w,h))
M = cv2.getRotationMatrix2D((w/2,h/2), 45, 1.0)
rotated = cv2.warpAffine(img, M, (w,h))
H = cv2.getPerspectiveTransform(src4, dst4)
warped = cv2.warpPerspective(img, H, (w,h))
```

Edges and Features

```
blur = cv2.GaussianBlur(gray, (5,5), 1.0)
edges = cv2.Canny(blur, 50, 150)
gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
mag = np.sqrt(gx**2 + gy**2)
sift = cv2.SIFT_create()
kp, des = sift.detectAndCompute(gray, None)
```

Segmentation and Morphology

```
_, otsu = cv2.threshold(gray, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU)
adapt = cv2.adaptiveThreshold(gray, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, (30,50,50), (60,255,255))
k = np.ones((3,3), np.uint8)
opened = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k)
closed = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k)
```

Hough

```
lines = cv2.HoughLinesP(edges, 1, np.pi/180, 80,
    minLineLength=50, maxLineGap=10)
circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, 1.2, 30,
    param1=100, param2=30,
    minRadius=10, maxRadius=100)
```

Exam Answer Templates

Theory Question Template

16. Give a one-sentence definition.
17. State the core mathematical idea or formula.
18. Explain the algorithm steps in order.
19. Mention important parameters and their effects.
20. State when to use it and one limitation.
21. If relevant, compare it with the closest alternative.

Coding Question Template

- # 1. Imports
- # 2. Read and validate image
- # 3. Convert required color space
- # 4. Preprocess if needed
- # 5. Apply the requested method
- # 6. Convert BGR to RGB only for Matplotlib display
- # 7. Display or save the result
- # 8. Mention what each key parameter controls

Three Likely Long-Question Patterns

Question pattern	What a complete answer must contain
Apply enhancement	Formula, float conversion, clipping, code, gamma/log choice.
Apply transformation	Matrix, output size, interpolation, warpAffine vs warpPerspective.
Extract or segment	Preprocessing, algorithm steps, parameters, output meaning, failure cases.

Final Night Revision Order

22. BGR/RGB, shape order, image coordinates, and basic operations.
23. Log, gamma, piecewise transformations, histogram equalization, and CLAHE.
24. Affine versus projective matrices and preservation rules.
25. Sobel versus Canny versus LoG; HOG versus LBP.
26. Thresholding, morphology, watershed, and K-means.
27. Hough line/circle parameters; SIFT descriptor, ratio test, and RANSAC.

Final advice Because the exam is open book but closed internet, tab the pages for Photometric Transformations, Geometric Transformations, Segmentation, SIFT Matching, and the Printable Code Cheat Sheet. Practice writing at least one full program without copying.